

JOBSHEET UAS



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

16.1 Persiapan

16.1.1 Langkah-langkah Percobaan

Class BinaryTree:

```
package JobsheetUAS;

public class BinaryTree10 {
    Node10 root;

    public BinaryTree10() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa10 mahasiswa) {
        Node10 newNode = new Node10(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node10 current = root;
            Node10 parent = null;
            while (true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }

    public void searchByName(String nama) {
        searchByName(root, nama);
    }

    private void searchByName(Node10 node, String nama) {
```

```

    if (node == null) {
        return;
    }

    if (node.mahasiswa.nama.equalsIgnoreCase(nama)) {
        node.mahasiswa.tampilInformasi();
        return;
    }

    searchByName(node.left, nama);
    searchByName(node.right, nama);
}

public void searchNPrintByClass(String kelas) {
    searchNPrintByClass(root, kelas);
}

private void searchNPrintByClass(Node10 node, String kelas) {
    if (node == null) {
        return;
    }

    searchNPrintByClass(node.left, kelas);

    if (node.mahasiswa.kelas.equalsIgnoreCase(kelas)) {
        node.mahasiswa.tampilInformasi();
    }

    searchNPrintByClass(node.right, kelas);
}

public void traversePreOrder(Node10 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

public void traverseInOrder(Node10 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

public void traversePostOrder(Node10 node) {
    if (node != null) {
        traversePostOrder(node.left);

```

```

        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

```

```

public Node10 getSuccessor(Node10 del) {
    Node10 successor = del.right;
    Node10 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

```

```

public void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    // cari node (current) yang akan dihapus
    Node10 parent = root;
    Node10 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }

    // penghapusan
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    } else {
        // jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {

```

```

        root = null;
    } else {
        if (isLeftChild) {
            parent.left = null;
        } else {
            parent.right = null;
        }
    }
} else if (current.left == null) { // jika hanya punya 1 anak (kanan)
    if (current == root) {
        root = current.right;
    } else {
        if (isLeftChild) {
            parent.left = current.right;
        } else {
            parent.right = current.right;
        }
    }
} else if (current.right == null) { // jika hanya punya 1 anak (kiri)
    if (current == root) {
        root = current.left;
    } else {
        if (isLeftChild) {
            parent.left = current.left;
        } else {
            parent.right = current.left;
        }
    }
} else { // jika punya 2 anak
    Node10 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}

public void addRekursif(Mahasiswa10 mahasiswa) {
    root = addRekursif(root, mahasiswa);
}

```

```

public Node10 addRekursif(Node10 current, Mahasiswa10 mahasiswa) {
    if (current == null) {
        return new Node10(mahasiswa);
    }

    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursif(current.left, mahasiswa);
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {
        current.right = addRekursif(current.right, mahasiswa);
    }
    return current;
}

public void cariMinIPK() {
    if (root == null) {
        System.out.println("Tree kosong");
        return;
    }

    Node10 current = root;

    while (current.left != null) {
        current = current.left;
    }

    current.mahasiswa.tampilInformasi();
}

public void cariMaxIPK() {
    if (root == null) {
        System.out.println("Tree kosong");
        return;
    }

    Node10 current = root;

    while (current.right != null) {
        current = current.right;
    }

    current.mahasiswa.tampilInformasi();
}

private int hitungTop = 0;

public void cari3IPKTertinggi() {
    hitungTop = 0;
    cari3IPKTertinggi(root);
}

```

```

private void cari3IPKTertinggi(Node10 node) {
    if (node == null || hitungTop >= 3) {
        return;
    }

    cari3IPKTertinggi(node.right);

    if (hitungTop < 3) {
        node.mahasiswa.tampilInformasi();
        hitungTop++;
    }

    cari3IPKTertinggi(node.left);
}

public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
    tampilMahasiswaIPKdiAtas(root, ipkBatas);
}

public void tampilMahasiswaIPKdiAtas(Node10 node, double ipkBatas) {
    if (node != null) {
        tampilMahasiswaIPKdiAtas(node.left, ipkBatas);

        if (node.mahasiswa.ipk > ipkBatas) {
            node.mahasiswa.tampilInformasi();
        }

        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
    }
}
}

```

Class BinaryTreeMain:

```

package JobsheetUAS;

import java.util.Scanner;

public class BinaryTreeMain10 {
    static void tampilMenu() {
        System.out.println("===== MENU =====");
        System.out.println("1. Cari berdasarkan nama");
        System.out.println("2. Cari top 3 IPK tertinggi");
        System.out.println("3. Cari & tampilkan mahasiswa dari kelas");
        System.out.print("Masukkan pilihan: ");
    }

    static void tambahData(BinaryTree10 bst) {
        bst.add(new Mahasiswa10("220101001", "Andi", "TI-1A", 3.95));
    }
}

```

```

        bst.add(new Mahasiswa10("220101002", "Budi", "TI-1A", 3.20));
        bst.add(new Mahasiswa10("220101003", "Sinta", "TI-1B", 3.82));
        bst.add(new Mahasiswa10("220101004", "Rina", "TI-1B", 3.45));
        bst.add(new Mahasiswa10("220101005", "Dimas", "TI-1C", 2.75));
        bst.add(new Mahasiswa10("220101006", "Fajar", "TI-1C", 3.10));
        bst.add(new Mahasiswa10("220101007", "Nabila", "TI-1D", 3.76));
        bst.add(new Mahasiswa10("220101008", "Rizky", "TI-1D", 3.50));
        bst.add(new Mahasiswa10("220101009", "Aulia", "TI-1E", 3.88));
        bst.add(new Mahasiswa10("220101010", "Kevin", "TI-1E", 3.00));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BinaryTree10 bst = new BinaryTree10();

        tambahData(bst);

        int pil;
        do {
            tampilMenu();
            pil = sc.nextInt();
            sc.nextLine();

            switch (pil) {
                case 1:
                    System.out.print("Masukkan nama: ");
                    String nm = sc.nextLine();

                    bst.searchByName(nm);
                    System.out.println();
                    break;

                case 2:
                    System.out.println("=== Top 3 IPK Tertinggi ===");
                    bst.cari3IPKTertinggi();
                    System.out.println();
                    break;

                case 3:
                    System.out.print("Masukkan kelas: ");
                    String kelas = sc.nextLine();

                    System.out.println("=== List Mahasiswa dari kelas " + kelas + "
                    ===");

                    bst.searchNPrintByClass(kelas);
                    System.out.println();
                    break;

                default:
                    System.out.println("Pilihan tidak valid!\n");
            }
        } while (pil != 0);
    }
}

```

```

        break;
    }
    } while (pil != 0);
    sc.close();
}
}

```

Class Mahasiswa:

```

package JobsheetUAS;

public class Mahasiswa10 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa10() {
    }

    Mahasiswa10(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampilInformasi() {
        System.out.println("NIM: " + nim + " " +
            "Nama: " + nama + " " +
            "Kelas: " + kelas + " " +
            "IPK: " + ipk);
    }
}

```

Class Node:

```

package JobsheetUAS;

public class Node10 {
    Mahasiswa10 mahasiswa;
    Node10 left, right;

    public Node10() {
    }

    public Node10(Mahasiswa10 mahasiswa) {
        this.mahasiswa = mahasiswa;
        left = right = null;
    }
}

```

}

16.1.2 Hasil Percobaan

```
===== MENU =====
1. Cari berdasarkan nama
2. Cari top 3 IPK tertinggi
3. Cari & tampilkan mahasiswa dari kelas
Masukkan pilihan: 1
Masukkan nama: andi
NIM: 220101001 Nama: Andi Kelas: TI-1A IPK: 3.95
```

```
===== MENU =====
1. Cari berdasarkan nama
2. Cari top 3 IPK tertinggi
3. Cari & tampilkan mahasiswa dari kelas
Masukkan pilihan: 2
=== Top 3 IPK Tertinggi ===
NIM: 220101001 Nama: Andi Kelas: TI-1A IPK: 3.95
NIM: 220101009 Nama: Aulia Kelas: TI-1E IPK: 3.88
NIM: 220101003 Nama: Sinta Kelas: TI-1B IPK: 3.82
```

```
===== MENU =====
1. Cari berdasarkan nama
2. Cari top 3 IPK tertinggi
3. Cari & tampilkan mahasiswa dari kelas
Masukkan pilihan: 3
Masukkan kelas: ti-1a
=== List Mahasiswa dari kelas ti-1a ===
NIM: 220101002 Nama: Budi Kelas: TI-1A IPK: 3.2
NIM: 220101001 Nama: Andi Kelas: TI-1A IPK: 3.95
```